Chapter 2. Understanding Virtual Threads

> *The price of reliability is the pursuit of the utmost simplicity. It is a price which the very rich find most hard to pay.*
>
> —Tony Hoare

Virtual threads are a groundbreaking addition to the Java concurrency toolkit that are fundamentally changing how developers write concurrent programs, making it practical to use threads as the primary unit of concurrency at massive scale. As we will discuss at length in this chapter, virtual threads differ significantly from the platform threads, or classical threads, that have served us over the years. In particular, while platform threads are managed by the underlying operating system or by thread libraries like those from POSIX, virtual threads are lightweight threads managed by the Java Virtual Machine (JVM) itself. This shift from OS-managed to JVM-managed threads represents more than just an implementation detail—it enables applications to create millions of threads without the memory and performance penalties that made such designs impractical with traditional threads. In this chapter, we'll take a deep dive into virtual threads by reviewing their architecture, discussing how they differ from platform threads, examining the motivation behind their creation for modern Java applications, and exploring how they simplify concurrent programming while delivering unprecedented scalability.

What Is a Virtual Thread?

Virtual threads are managed by the JVM, which allows them to operate more efficiently than traditional threads that rely on the operating system for scheduling and management. Virtual threads are executed on top of carrier threads, which are essentially threads from the Fork/Join Pool. This design allows virtual threads to inherit the benefits of advanced thread pooling mechanisms and efficient work-stealing algorithms.

It's important to note that the virtual thread scheduler implemented within the JVM is based on a work-stealing `ForkJoinPool`, operating in a First-In, First-Out (FIFO) mode. This `ForkJoinPool` serves as the foundation for scheduling virtual threads and is distinct from the common pool used for other purposes, such as the implementation of parallel streams, which operate in a Last-In, First-Out (LIFO) mode.

The parallelism of the virtual thread scheduler, which refers to the number of platform threads available for scheduling virtual threads, is a configurable parameter. By default, it is set to the number of available processors on the system, ensuring optimal utilization of hardware re-

sources. However, developers can fine-tune this parameter using the `jdk.virtualThreadScheduler.parallelism` system property, allowing them to adjust the level of parallelism based on their specific application requirements and workload characteristics.

You can set system properties in Java using the `-D` command-line option when you start your Java application, like this:

```
java -Djdk.virtualThreadScheduler.parallelism=4 -jar yourApp.jar
```

In this example, the `jdk.virtualThreadScheduler.parallelism` system property is set to 4. Replace *`yourApp.jar`* with the actual name of your Java application.

In code, you can use the `System.setProperty(key, value)` method:

```
System.setProperty("jdk.virtualThreadScheduler.parallelism", "4");
```

Please keep in mind that these settings need to be adjusted before they are used for the first time; this is typically done at the beginning of the main method or even before your Spring/Jakarta EE/Quarkus application context starts up.

The Two Kinds of Threads in Java

With the introduction of virtual threads, Java now features two distinct kinds of threads: platform threads and virtual threads (Figure 2-1).

Platform threads

: These are the threads native to Java since its inception, which explains why they're sometimes called *traditional* or *classical threads*. You might also encounter terms like *native threads* or *OS threads* in different contexts. In the Java Development Kit (JDK), they are officially called *platform threads*. A platform thread is a heavyweight thread that is executed through the underlying operating system, relying on the OS for thread scheduling and management. Platform threads maintain a one-to-one mapping between Java threads and kernel threads, which are managed by the operating system. These threads utilize the operating system's existing scheduling and context-switching mechanisms. Platform threads have been the foundation of Java's concurrent programming model since the language's inception.

Virtual threads

: Virtual threads, sometimes called *user-mode threads* or *lightweight threads*, are a new addition to Java's concurrency model since JDK 21. Unlike platform threads, virtual threads are managed entirely by the JVM. Millions of virtual threads can be created without exhausting system resources. Virtual threads are not directly mapped to kernel threads. Instead, multiple virtual threads share a smaller pool of carrier threads (which are platform threads), allowing the JVM to efficiently multiplex many virtual threads onto relatively few OS resources.

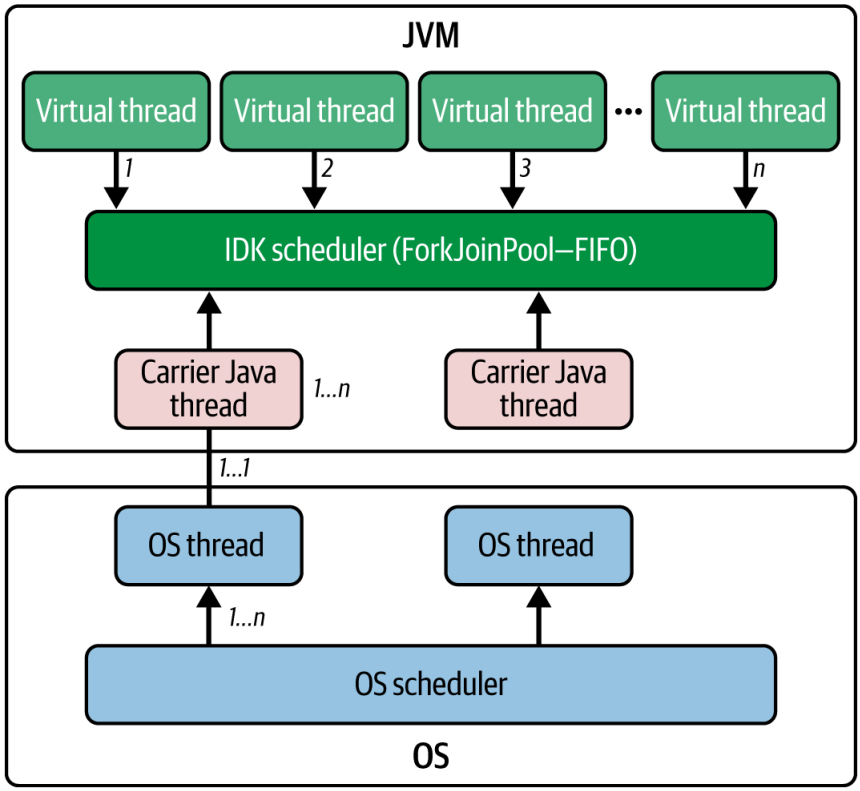

Figure 2-1. Internals of thread scheduling: virtual threads, carrier threads

Key Differences from Platform Threads

Now that we know there are two kinds of threads—platform threads and virtual threads—let's explore some of the key differences between them:

Lightweight

Virtual threads use much less memory and fewer resources than platform threads, so you can create millions of virtual threads without exhausting machine resources.

Scheduling

Virtual threads are scheduled by the JVM, not by the operating system, making better use of CPU cycles and bypassing any overhead incurred by an operating system thread scheduler.

Blocking tolerance

Virtual threads don't come with a performance penalty when they need to do blocking operations, such as read from the console, read from a file or network connection, write to a file or network, or sleep. That's because when a virtual thread encounters a blocking operation, it yields control back to the carrier thread, allowing other virtual threads to continue executing efficiently.

Seamless integration

Virtual threads are designed to seamlessly integrate with existing codebases. Developers can continue to use coding patterns and abstractions that are familiar to them, without having to adjust their workflows.

Given that we cannot fix the limitations of platform threads and can only do so much to scale them, virtual threads present a more scalable and performant concurrency model—one that we believe can and will transform how Java developers think about concurrent programming. Over the course of this book, in this and subsequent chapters, we will dive

deeper into the details of how virtual threads work, look at how to use them in real-world projects, and offer example projects to help you get started with developing in this exciting new world.

Setting Up Your Environment for Virtual Threads

Project Loom's virtual threads are now a stable feature as of JDK 21. To start using them, ensure you have JDK 21 installed. For managing multiple JDK versions, we recommend SDKMAN. This versatile tool streamlines installation and allows for easy switching between versions. Refer to the official SDKMAN website for installation instructions. Installing a JDK with SDKMAN enables developers to easily manage Java versions and dependencies on their systems, facilitating Java development tasks.

After installing SDKMAN, use the following command to list available JDK versions (see Figure 2-2 for the result):

```
sdk list java
```

```
================================================================================
Available Java Versions for macOS ARM 64bit
================================================================================
 Vendor        | Use | Version      | Dist    | Status     | Identifier
--------------------------------------------------------------------------------
 Corretto      |     | 21.0.2       | amzn    |            | 21.0.2-amzn
               |     | 21.0.1       | amzn    |            | 21.0.1-amzn
               |     | 17.0.10      | amzn    |            | 17.0.10-amzn
               |     | 17.0.9       | amzn    |            | 17.0.9-amzn
               |     | 11.0.22      | amzn    |            | 11.0.22-amzn
               |     | 11.0.21      | amzn    |            | 11.0.21-amzn
               |     | 8.0.402      | amzn    |            | 8.0.402-amzn
               |     | 8.0.392      | amzn    |            | 8.0.392-amzn
 Gluon         |     | 22.1.0.1.r17 | gln     |            | 22.1.0.1.r17-gln
               |     | 22.1.0.1.r11 | gln     |            | 22.1.0.1.r11-gln
 GraalVM CE    |     | 21.0.2       | graalce |            | 21.0.2-graalce
               |     | 21.0.1       | graalce |            | 21.0.1-graalce
               |     | 17.0.9       | graalce |            | 17.0.9-graalce
 GraalVM Oracle|     | 21.0.2       | graal   |            | 21.0.2-graal
               |     | 21.0.1       | graal   |            | 21.0.1-graal
               |     | 17.0.10      | graal   |            | 17.0.10-graal
               |     | 17.0.9       | graal   |            | 17.0.9-graal
 Java.net      |     | 23.ea.12     | open    |            | 23.ea.12-open
               |     | 23.ea.11     | open    |            | 23.ea.11-open
               |     | 23.ea.10     | open    |            | 23.ea.10-open
               |     | 23.ea.8      | open    | installed  | 23.ea.8-open
               |     | 23.ea.7      | open    |            | 23.ea.7-open
               |     | 23.ea.6      | open    |            | 23.ea.6-open
               |     | 23.ea.5      | open    |            | 23.ea.5-open
               |     | 23.ea.4      | open    |            | 23.ea.4-open
               |     | 23.ea.3      | open    |            | 23.ea.3-open
               |     | 23.ea.2      | open    |            | 23.ea.2-open
               |     | 23.ea.1      | open    |            | 23.ea.1-open
               |     | 22.ea.36     | open    |            | 22.ea.36-open
               |     | 22.ea.35     | open    |            | 22.ea.35-open
               |     | 22.ea.34     | open    |            | 22.ea.34-open
```

Figure 2-2. SDKMAN list command output for available Java versions

To install JDK 21 (if you don't have it already), use:

```
sdk install java <YOUR_FAVORITE_JDK_DISTRIBUTION>
```

For example, if you use OpenJDK, then you'll use:

```
sdk install java 21.0.2-open
```

NOTE

This chapter uses JDK 21, which remains the most widely adopted version for virtual threads in production environments. In subsequent chapters, we'll switch to the latest JDK versions to explore newer features such as:

- Scoped values
- Structured concurrency

Creating Virtual Threads in Java

Now that we have JDK 21 installed, let's explore ways to create virtual threads in Java.

For simple use cases, the `Thread.startVirtualThread()` method offers a direct approach. It takes a `Runnable`, so we can pass a lambda expression, and whatever we pass through it will execute. For example:

```
Thread.startVirtualThread(() -> {
        System.out.println("Unleash massive parallelism with virtual
        threads! Here's a taste.");
  });
```

If you run the preceding code, you might expect to see a message on your console. However, nothing will be printed! Why? Virtual threads in Java are daemon threads by default. This means that when the main thread (the one that created the virtual thread) finishes, the JVM terminates any remaining daemon threads.

To ensure your virtual thread's task completes, you need to wait for it to finish:

```
public static void main(String[] args) throws InterruptedException {
    Thread vThread = Thread.startVirtualThread(() -> {
        System.out.println("Virtual threads make concurrency effortless!" +
            "See for yourself.");
    });
    vThread.join();
}
```

For more control over thread creation, you can use the Thread Builder API:

```
var startedThread = Thread.ofVirtual()
    .start(() -> System.out.println("Hello world!"));
startedThread.join();
```

To create a thread without starting it immediately, use this code:

```
var unstartedThread = Thread.ofVirtual()
    .unstarted(() -> System.out.println("Hello world!"));
```

```
// Start the thread later when needed
unstartedThread.start();
```

If your application already uses executors, you can smoothly transition to virtual threads:

```
try (var virtualExecutor = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<String> future = virtualExecutor.submit(this::callService);
    // Process the future result
}
```

This is especially helpful in large projects where immediate refactoring for virtual threads might be impractical.

Having these different approaches allows us to seamlessly integrate virtual threads into our existing codebase, taking advantage of the improved performance, scalability, and resource efficiency offered by this new concurrency model while maintaining familiar coding patterns and abstractions.

Adapting to Virtual Threads

The introduction of virtual threads brought necessary changes to Java's Thread API, but in a way that optimizes for developer familiarity. A virtual thread in Java is essentially an instance of the `Thread` class, and cancellation works the same way as it does for platform threads: by invoking the `interrupt()` method. The code running within the thread must either check the `interrupted` flag or call methods that handle interruption automatically (which most blocking methods do).

Let's see an example of thread interruption in platform and virtual threads.

Platform thread interruption:

```
public class PlatformThreadInterruption {
    public static void main(String[] args) {
        Thread platformThread = Thread.ofPlatform().start(() -> { ❶
            try {
                System.out.println("Platform thread started...");
                for (int i = 0; i < 5; i++) {
                    System.out.println("Platform thread working: " + i);
                    Thread.sleep(1000); // Simulate work
                }
                System.out.println("Platform thread finished.");
            } catch (InterruptedException e) { ❷
                System.out.println("Platform thread interrupted!");
                // Handle cleanup if needed
            }
        });
        try {
            Thread.sleep(2500);  // Let the thread run for a bit ❸
        } catch (InterruptedException e) {}
```

```
            platformThread.interrupt(); ❹
        }
    }
```

Virtual thread interruption:

```
public class VirtualThreadInterruption {
    public static void main(String[] args) {
        Thread virtualThread = Thread.ofVirtual().start(() -> { ❶
            try {
                System.out.println("Virtual thread started...");
                for (int i = 0; i < 5; i++) {
                    System.out.println("Virtual thread working: " + i);
                    Thread.sleep(1000); // Automatically yields
                                        to other virtual threads
                }
                System.out.println("Virtual thread finished.");
            } catch (InterruptedException e) { ❷
                System.out.println("Virtual thread interrupted!");
                // Handle cleanup if needed
            }
        });
        try {
            Thread.sleep(2500);  // Let the thread run for a bit ❸
        } catch (InterruptedException e) {}
        virtualThread.interrupt(); ❹
    }
}
```

Let's understand the interruption flow in both examples:

❶ Creates either a platform or virtual thread using the builder pattern.

❷ Catches `InterruptedException` when the thread is interrupted during a blocking operation.

❸ Main thread waits 2.5 seconds, allowing the worker thread to complete two to three iterations.

❹ Interrupts the worker thread, triggering the exception and cleanup.

If we run the above code, we will get following outputs:

```
java PlatformThreadInterruption.java
Platform thread started...
Platform thread working: 0
Platform thread working: 1
Platform thread working: 2
Platform thread interrupted!
```

and

```
java VirtualThreadInterruption.java

Virtual thread started...
Virtual thread working: 0
```

```
Virtual thread working: 1
Virtual thread working: 2
Virtual thread interrupted!
```

Both examples produce identical output, demonstrating that virtual threads maintain the same interruption semantics as platform threads. The key difference is that virtual-thread blocking operations (like `Thread.sleep()`) automatically yield the underlying carrier thread to other virtual threads.

While virtual threads maintain familiar APIs, they have some distinct characteristics that reflect their lightweight, managed nature, for example, thread groups.

All virtual threads belong to a single thread group; there is no API to create a virtual thread with a different thread group. If we create a virtual thread and invoke the `getThreadGroup()` method, we will always get instances of the same `ThreadGroup`. For example:

```
public class VirtualThreadGroupExample {
    public static void main(String[] args) throws InterruptedException {
        Set<ThreadGroup> threadGroups = new HashSet<>();

        for (int i = 0; i < 100; i++) { ❶
            Thread vThread = Thread.ofVirtual().start(() -> {
                try {
                    Thread.sleep(10);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
            threadGroups.add(vThread.getThreadGroup()); ❷
        }
        Thread.sleep(1000); // Wait for threads to complete
        System.out.println("Unique thread groups: " + threadGroups.size());
        System.out.println("Thread group: " + threadGroups.iterator().next());
    }
}
```

In the preceding code, what we have done is:

❶ Create 100 virtual threads to test thread group assignment

❷ Collect thread groups from all virtual threads into a set

When we run the preceding code, we will get the following output:

```
Unique thread groups: 1
Thread group: java.lang.ThreadGroup[name=VirtualThreads,maxpri=10]
```

From the preceding output, we see virtual threads have several immutable characteristics that simplify their management:

Priority

Virtual threads have a priority level set to `NORM_PRIORITY`.

Daemon status

All virtual threads are daemon threads by default. Any attempt to change this status using `setDaemon` has no effect.

Thread priority

Calling `setPriority` on a virtual thread does not change its priority.

The Thread API has also been updated to include some new methods and deprecate others:

Is Virtual

The `Thread::isVirtual` instance method helps you determine if a thread is virtual.

Duration-based methods

Java 19 introduces `join(Duration)` and `sleep(Duration)` instance methods, which are unrelated to virtual threads but add convenience.

Thread ID

The nonfinal `getId()` method is deprecated, and it is recommended to use the final `threadId()` method instead.

As for deprecated methods, since Java 20, the `stop()`, `suspend()`, `resume()`, and `countStackFrames()` methods will throw an `UnsupportedOperationException` for both platform and virtual threads, as they are inherently unsafe, as mentioned in the Java Documentation. These methods have been deprecated since Java 1.2 and are scheduled for removal.

Virtual threads have some minor limitations, however. It's worth noting that the static `Thread::getAllStackTraces` method only returns stack traces of platform threads, excluding virtual threads. Also, there is currently no way to find out which platform thread is executing a given virtual thread.

While virtual threads introduce some changes and limitations to the Thread API, the broad similarity to the existing API and threading concepts makes the transition to virtual threads largely invisible to developers. Using the new capabilities within the updated guidelines, developers can continue with their existing codebase but will now reap the reward of virtual-thread performance and scalability.

Demonstrating Virtual Thread Creation in Java

Virtual threads unleash the true scalability potential of Java applications. By using these virtual threads, you can make a Java application dramatically more scalable because you can have many more tasks running in parallel. It's crucial for modern server applications to handle thousands of concurrent requests. We'll now take a look at virtual threads in action.

Consider the following example where we submit 10,000 tasks using an executor that creates a new virtual thread for each task:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10_000).forEach(i -> {
        executor.submit(() -> {
            Thread.sleep(Duration.ofSeconds(1));
            return i;
        });
    });
}
```

The tasks are simple: each just sleeps for one second. This is easily achievable on modern hardware with 10,000 virtual threads running concurrently. What's impressive is that the JDK manages this with a minimal number of OS threads, potentially even just one.

By contrast, if we used `Executors.newCachedThreadPool()`, which creates a new platform thread for each task, the program might crash due to the overhead of creating 10,000 OS threads. Similarly, using a fixed thread pool like `Executors.newFixedThreadPool(200)` would drastically limit concurrency. With only 200 platform threads, many tasks would run sequentially, causing the program to take significantly longer to complete.

Throughput and Scalability

Virtual threads can achieve a throughput of about 10,000 tasks per second after sufficient warm-up. If you increase the number of tasks to 1,000,000, the program still runs smoothly, achieving a throughput of nearly 1,000,000 tasks per second.

It's important to clarify that virtual threads are not designed to be faster but to offer greater scalability. They follow Little's Law, providing higher throughput by enabling greater concurrency, not by executing tasks more quickly.

Under ideal circumstances, virtual threads can substantially enhance throughput in applications with the following characteristics:

High number of concurrent tasks

: Virtual threads can be a game-changer if your application has more than a few thousand concurrent tasks. This is ideal for high-volume web servers needing to handle thousands of requests simultaneously or for applications performing many I/O-bound operations in parallel.

Non-CPU-bound workload

: Virtual threads are particularly beneficial when tasks spend more time waiting (e.g., for I/O operations) than they do performing CPU-bound operations.

This is an important caveat: While virtual threads introduce a new level of scalability to Java applications, they are not a cure-all for concurrency.

They're likely to be most effective when we have lots of concurrent tasks coupled with lightweight non-CPU-bound workloads. Compared with traditional multithreading, virtual threads can provide a new dimension of scalability to Java applications. This allows developers to build highly concurrent and scalable applications that can handle a vast number of requests without taxing our CPU cycles or memory.

The Fundamental Principle Behind Virtual Threads' Scalability

Little's Law is a fundamental principle that provides valuable insights into the performance of queuing systems, including multithreaded applications. It establishes a mathematical relationship between latency, concurrency, and throughput, three key elements that are intrinsically tied to the performance of any computing system.

The law is elegantly simple, stating that for a stable system, the throughput (λ) is given by:

$$\lambda = \frac{N}{d}$$

Let's delve into the components of Little's Law and learn about their significance:

Throughput (λ)

: The average number of items (e.g., tasks, requests) completed per unit of time

Concurrency (N)

: The average number of items being processed simultaneously

Response time (d)

: The average time it takes for a single item to be processed from start to finish

What's remarkable about Little's Law is its agnosticism toward the nature of the system it describes. It doesn't differentiate between time spent "doing work" versus "waiting," nor does it care about what constitutes the unit of concurrency—be it a thread, a CPU core, an ATM machine, or even a human bank teller. It simply offers a formula to scale up throughput.

Virtual threads and Little's Law

Traditional threading models often hit a wall due to OS-level limitations on the number of threads that can be spawned. When this happens, the system's throughput becomes constrained by Little's Law. The law shows us that if we can't increase N (concurrency), we're left with only one other variable to manipulate: d (latency). However, reducing latency is not always within a developer's control, especially in I/O-bound tasks.

This is where virtual threads come into the picture. They offer a way out of this limitation by allowing for a much higher N—that is, they enable higher concurrency without requiring a change in the programming model. This is a significant advantage because it directly leads to higher

throughput, as indicated by Little's Law. Virtual threads offer a way to scale up *N* without proportionally scaling down *d*.

To illustrate this concept, let's craft a code example that allows us to measure and demonstrate how Little's Law's principle of throughput relates to virtual threads and their scalability benefits:

```
import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicLong;
import java.util.stream.IntStream;

public class LittleLawExample {
  public static void main(String[] args) {
    int numTasks = 10000; ❶
    int avgResponseTimeMillis = 500; // Average task response time ❷
    // Simulate adjustable I/O-bound work
    Runnable ioBoundTask = () -> {
      try {
        Thread.sleep(Duration.ofMillis(avgResponseTimeMillis)); ❸
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    };

    System.out.println("=== Little's Law Throughput Comparison ===");
    System.out.println("Testing " + numTasks + " tasks with "
                       + avgResponseTimeMillis + "ms latency each\n");
    benchmark("Virtual Threads",
        Executors.newVirtualThreadPerTaskExecutor(), ioBoundTask, numTasks);
    benchmark("Fixed ThreadPool (100)",
        Executors.newFixedThreadPool(100), ioBoundTask, numTasks);
    benchmark("Fixed ThreadPool (500)",
        Executors.newFixedThreadPool(500), ioBoundTask, numTasks);
    benchmark("Fixed ThreadPool (1000)",
        Executors.newFixedThreadPool(1000), ioBoundTask, numTasks);
  }

  static void benchmark(String type, ExecutorService executor, Runnable task,
                        int numTasks) {
    Instant start = Instant.now(); ❹
    AtomicLong completedTasks = new AtomicLong();
    try (executor) { ❺
      IntStream.range(0, numTasks)
          .forEach(i -> executor.submit(() -> {
            task.run();
            completedTasks.incrementAndGet(); ❻
          }));
    } ❼
    Instant end = Instant.now();
    long duration = Duration.between(start, end).toMillis();
    // Tasks per second
    double throughput = (double) completedTasks.get() / duration * 1000; ❽
    System.out.printf("%-25s - Time: %5dms, Throughput: %8.2f tasks/s%n",
        type, duration, throughput);
  }
}
```

Let's explore what we have accomplished in this program:

❶ Ten thousand tasks provide sufficient load to demonstrate throughput differences while keeping execution time manageable.

❷ A response time of 500ms simulates realistic I/O operations like database queries or API calls, representing the *d* (latency) component in Little's Law.

❸ `Thread.sleep()` mimics blocking I/O operations. With virtual threads, this automatically yields the carrier thread to other virtual threads.

❹ Measures total execution time from task submission start to completion of all tasks.

❺ `Try` -with-resources ensures proper executor shutdown and waits for all submitted tasks to complete.

❻ `AtomicLong` safely tracks completed tasks across all concurrent threads.

❼ When the `try` block exits, all submitted tasks have finished executing.

❽ Converts completed tasks per total time into tasks per second for easy comparison.

In this example, we simulate an I/O-bound task with an average response time of 500 milliseconds. We then benchmark the throughput achieved by virtual threads and compare it with fixed thread pools of varying sizes, which represent the traditional threading model.

When we execute the preceding code, we get results demonstrating virtual threads' dramatic advantage:

```
=== Little's Law Throughput Comparison ===
Testing 10000 tasks with 500ms latency each
Virtual Threads           - Time:   552ms, Throughput: 18115.94 tasks/s
Fixed ThreadPool (100)    - Time: 50381ms, Throughput:   198.49 tasks/s
Fixed ThreadPool (500)    - Time: 10106ms, Throughput:   989.51 tasks/s
Fixed ThreadPool (1000)   - Time:  5080ms, Throughput:  1968.50 tasks/s
```

As you can see from the results, virtual threads outperform traditional thread pools in terms of throughput for this I/O-bound scenario. Virtual threads can achieve a throughput of around 18115.94 tasks per second, while even a large fixed thread pool of 1,000 threads can only manage a throughput of around 1,968.50 tasks per second.

TIP

The previous example provides a basic demonstration of Little's Law with virtual threads. For more rigorous performance analysis in production environments, consider these enhancements:

- JVM warm-up runs to account for just-in-time compilation effects
- Multiple benchmark iterations with statistical analysis (mean, standard deviation)
- Memory usage monitoring to understand resource trade-offs
- Realistic I/O simulation with latency variance (±10–20%) to model real-world conditions
- Scalability testing across different task volumes to identify breaking points
- Timeout protection and proper error handling for robust measurements

These improvements help distinguish between genuine performance differences and measurement artifacts, providing more reliable insights for production decision-making. For critical performance evaluations, consider using established microbenchmarking frameworks like Java Microbenchmark Harness (JMH).

With platform threads, throughput can improve with larger thread pools, but there's a limit. Eventually, OS resources will be constrained, and thread management overhead becomes the bottleneck.

This example clearly demonstrates how virtual threads can leverage Little's Law by enabling a higher value of N (concurrency) without the need to scale down d (latency) proportionally. Virtual threads can achieve superior throughput by allowing for a significantly higher number of concurrent tasks, particularly in I/O-bound scenarios where reducing latency is not a viable option.

The Practical Implications

Virtual threads free developers from having to tweak or even fundamentally rethink their programming models to achieve more throughput. If an application has many mostly I/O-bound tasks that spend a lot of time waiting, then we can substantially increase throughput without having to change how those tasks are written.

By exploiting virtual threads, we can, in effect, increase N in Little's Law, which in turn increases concurrency and, therefore, throughput. This is particularly useful when the latency d is hard to reduce due to the nature of the work (e.g., network I/O or disk I/O).

It's important to note, however, that virtual threads are not a silver bullet for all performance issues. In situations where I/O is the bottleneck, virtual threads can offer dramatic improvements in throughput; however, virtual threads aren't going to help in situations where the bottleneck is not the number of concurrent tasks that are active but instead the amount of available computational throughput.

When developers grasp Little's Law and its implications for virtual threads, they'll have a better sense of when and how virtual threads can be used to make their applications more scalable and faster.

How Virtual Threads Work Under the Hood

Virtual threads offer a significant leap in Java's concurrency model. To understand their power, let's explore their core mechanisms.

Stack Frames and Memory Management

At the core of virtual threads is an alternative implementation of `java.lang.Thread` that stores its stack frames in Java's garbage-collected heap. In contrast, traditional threads store stack frames in monolithic memory blocks allocated by the operating system. This novel approach eliminates the need to estimate a thread's required stack size. A virtual thread's memory footprint starts from just a few hundred bytes, and it automatically adjusts as the call stack grows and shrinks. This dynamic memory management significantly improves resource efficiency.

Carrier Threads and OS Involvement

The operating system is unaware of virtual threads; it only recognizes platform threads, which remain the unit of OS-level scheduling. To execute code in a virtual thread, the Java runtime mounts it onto a platform thread, known as a *carrier thread*. These carrier threads are part of a specialized `ForkJoinPool`. This process involves temporarily copying the necessary stack frames from the heap to the carrier thread's stack. Essentially, the carrier thread is "borrowed" to run the virtual thread's code.

Handling Blocking Operations

One of the most consequential improvements is how virtual threads deal with blocking operations. When a virtual thread arrives at an operation that would typically block— perhaps it's waiting for I/O—it can be unmounted from its carrier thread. Its modified stack frames are copied back to the heap, and the carrier thread gets freed to go off and do other work. This functionality has been retrofitted to almost all blocking points in the JDK. It's what makes virtual threads highly efficient in resource utilization.

Transparency and Invisibility

The process of mounting and unmounting a virtual thread is transparent to the Java code. There is no way for the Java code to discern the identity of the current carrier thread. Even `ThreadLocal` values of the carrier thread are invisible to a virtual thread. This level of abstraction ensures that virtual threads can be used seamlessly without requiring changes to existing Java codebases.

The concept of virtual threads is reminiscent of virtual memory systems. In a virtual memory system, applications run under the illusion that they

have access to effectively unlimited amounts of address space. This is enabled by a hardware-level mapping between which part of virtual memory is actually memory, and which part is disk. Similarly, virtual threads create the illusion of effectively unlimited multithreading, by sharing scarce and costly platform threads. Inactive virtual thread stacks are "paged out" to the heap, just as unused memory pages in a virtual memory system are "paged out" to disk.

More about how virtual threads work and their internals will be discussed in later chapters.

Simplifying Asynchronous Operations

With virtual threads, asynchronous operations and task aggregation become a breeze. In Java, the burden of blocking while waiting for the asynchronous task to finish is effectively lifted. Developers can now happily call the blocking `get` on a `Future` and not have to take a performance hit. But where virtual threads really shine is when it comes to aggregating the results of multiple asynchronous tasks.

Consider a scenario where you need to fetch various components of a sentence from different APIs and combine them into a cohesive phrase. For instance, you might want to retrieve an adjective and a noun to create a simple yet randomly generated phrase. With virtual threads, this task becomes remarkably straightforward:

```
public String generatePhrase() throws ExecutionException, InterruptedException {
  try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<String> adjectiveFuture = executor.submit(this::fetchAdjective);
    Future<String> nounFuture = executor.submit(this::fetchNoun);
    String adjective = adjectiveFuture.get();
    String noun = nounFuture.get();
    return adjective + " " + noun;
    }
}
private String fetchAdjective() {
   // Fetch adjective from an API
   return "beautiful";
}
private String fetchNoun() {
   // Fetch noun from an API
   return "sunset";
}
```

In this example, we leverage the `VirtualThreadPerTaskExecutor`, a convenient utility for creating virtual threads for individual tasks. We submit two tasks, one to fetch an adjective and one for a noun, using the `ExecutorService`'s `submit` method.

The true elegance of virtual threads is revealed when we call `get` on the `Future` objects returned by the submitted tasks. Since virtual threads are lightweight and efficient, we can comfortably block and wait for the results without consuming excessive resources or causing performance degradation. This approach simplifies the handling of asynchronous op-

erations, eliminating the need for complex callback structures or intricate synchronization mechanisms.

Once we have the adjective and noun, we can seamlessly concatenate them to form a phrase, such as “beautiful sunset” in this case.

The `invokeAll` method can still be utilized even when there are similar tasks with the same result type. It is particularly helpful when the goal is to compile results into a list. For instance, you might want to filter different images in a similar manner:

```
package ca.bazlur.modern.concurrency.c02;

import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;
import java.util.List;
import java.util.concurrent.*;

public class ImageProcessingExample {

  public static void main(String[] args)
      throws InterruptedException, ExecutionException {
    try (var service = Executors.newVirtualThreadPerTaskExecutor()) { ❶

      List<Callable<BufferedImage>> tasks = List.of(
          () -> resize("https://example.com/img1.jpg", 200, 200),
          () -> grayscale("https://example.com/img2.jpg"),
          () -> rotate("https://example.com/img3.jpg", 90)
      ); ❷

      List<Future<BufferedImage>> results = service.invokeAll(tasks); ❸

      // Process and save transformed images
      int i = 1;
      for (Future<BufferedImage> future : results) { ❹
        BufferedImage image = future.get(); ❺
        ImageIO.write(image, "jpg",
            new File("output_image" + i + ".jpg"));
        i++;
      }
    } catch (IOException e) {
      throw new RuntimeException(e);
    }
  }

  static BufferedImage resize(String url, int width, int height) {
    //Logic to download and resize the image goes here
    return null;
  }

  static BufferedImage grayscale(String url) {
    // Logic to download and convert the image to
    // grayscale goes here
    return null;
  }

  static BufferedImage rotate(String url, double angle) {
```

```
        //Logic to download and rotate the image goes here
        return null;
    }
}
```

Let's see what we have here:

1. Uses the virtual thread executor, but now for multiple concurrent image processing tasks.
2. Creates a list of `Callable` tasks, each representing a different image transformation operation.
3. Submits all tasks simultaneously with `invokeAll()`; they execute concurrently on separate virtual threads.
4. Iterates through the results in the same order as the original task list.
5. Retrieves each result with blocking `get()` calls that efficiently yield virtual threads during waits.

Modern applications frequently demand the ability to handle multiple tasks simultaneously while aggregating their results efficiently. Virtual threads provide an elegant solution by enabling parallel execution with seamless result collection, offering significant advantages in contemporary computing environments where I/O-bound operations dominate application performance.

The Promise of Structured Concurrency

Although all the preceding examples are valid and involve fairly straightforward ways of dealing with concurrent tasks, they are somewhat simplistic—for instance, they do not handle exceptions or timeouts very well. It's possible that the `Future::get` method throws exceptions, and without a timeout mechanism, it could potentially block indefinitely.

Java's JEP 505 introduces structured concurrency, aiming to simplify how we write robust concurrent code. The API provides a structured scope within which tasks run, and if any of them fail or timeout, all tasks within the scope are automatically canceled. Here is a brief example:

```
public static void main(String[] args) {
    try (StructuredTaskScope scope = StructuredTaskScope.open()) {
        StructuredTaskScope.Subtask<String> subtask1
            = scope.fork(() -> fetchData("https://api1.example.com"));
        StructuredTaskScope.Subtask<String> subtask2 =
            scope.fork(() -> fetchData("https://api2.example.com"));
        scope.join();
        var result = subtask2.get() + subtask1.get();
        System.out.println(result);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

NOTE

The preceding code requires JDK 25 with preview features enabled (`--enable-preview`).

These fail-fast scopes also let you provide timeouts and make it very easy to encapsulate exceptions. With the fail-fast scope, `subtask1` and `subtask2` are forked tasks running within the scope. If either of them fails (by raising an exception) or if the timeout elapses, all tasks running within the scope are canceled, and the exception is propagated back. This results in cleaner and more robust code, brilliantly encapsulating exception propagation and timeouts within a built-in construct.

The introduction of structured concurrency in Java represents a significant step forward in simplifying the development of concurrent applications. By providing a structured scope for running tasks, the API enables automatic cancellation and exception handling, reducing the burden on developers of managing these aspects manually.

Moreover, structured concurrency promotes a more declarative style of programming, where developers can express their intentions clearly and concisely. Instead of dealing with low-level details, such as creating and managing threads or implementing complex cancellation logic, developers can focus on the higher-level tasks and let the Structured Concurrency API handle the underlying complexities.

This is just a glimpse of structured concurrency's potential. We'll explore this fascinating subject in greater depth in subsequent chapters, examining advanced patterns, different scope types, and how structured concurrency complements virtual threads to create truly robust concurrent applications.

Managing Resource Constraints with Rate Limiting

Virtual threads are essentially an open door to unlimited concurrency, offering the potential to handle an unprecedented number of tasks simultaneously. While we've discussed this as a positive thing so far, it can be challenging in certain aspects. This is because not all parts of the software can handle the same amount of load. For example, the web application that we are building can certainly accept a million requests, thus creating a million virtual threads, but the underlying database may not be able to handle that many requests.

Before virtual threads, the thread pool helped us limit requests without the need to implement additional mechanisms. So if the thread pool has 1,000 threads, the underlying resource could, at most, have 1,000 simultaneous loads, not more. However, this presents a conundrum since virtual threads can be virtually unlimited. So now the question becomes: How do you prevent overloading services when using virtual threads?

The answer lies in implementing rate-limiting mechanisms specific to the resource you are accessing. These mechanisms can range from simple to complex, depending on the nature of the resource and the service level agreements (SLAs) you must adhere to.

Let's explore an example using a semaphore to control the number of concurrent requests to a web service. Crucially, the focus here is on the principle of rate limiting:

```
import java.net.URI;
import java.net.http.*;
import java.time.Duration;
import java.time.Instant;
import java.util.List;
import java.util.concurrent.*;
import java.util.stream.IntStream;
public class ResourceAwareRateLimitExample {
  private static final HttpClient CLIENT = HttpClient.newBuilder()
      .connectTimeout(Duration.ofSeconds(10)) ❶
      .build();
  private static final int MAX_PARALLEL = 10; ❷
  private static final Semaphore gate = new Semaphore(MAX_PARALLEL); ❸
private static final String API_URL =
    "https://api.chucknorris.io/jokes/random";

  public static void main(String[] args) throws Exception {
    Instant start = Instant.now();
    List<String> jokes = fetchJokes(50); ❹
    long ms = Duration.between(start, Instant.now()).toMillis();
    System.out.printf("Fetched %d jokes in %d ms (avg %d ms)%n",
        jokes.size(), ms, ms / jokes.size());
    jokes.stream().limit(3).forEach(j -> System.out.println("• " + j));
  }

  private static List<String> fetchJokes(int n) throws Exception {
    try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) { ❺
      List<Future<String>> futures = IntStream.range(0, n)
          .mapToObj(i -> pool.submit(ResourceAwareRateLimitExample::fetchJoke))
          .toList();
      return futures.stream()
          .map(ResourceAwareRateLimitExample::join) ❻
          .toList();
    }
  }

  private static String fetchJoke() throws Exception {
    HttpRequest req = HttpRequest.newBuilder(URI.create(API_URL))
        .GET()
        .timeout(Duration.ofSeconds(30)) ❼
        .build();
    try {
      gate.acquire(); ❽
      HttpResponse<String> res
        = CLIENT.send(req, HttpResponse.BodyHandlers.ofString());
      if (res.statusCode() != 200)
        throw new RuntimeException("API error " + res.statusCode());
      return parseJoke(res.body());
    } finally {
      gate.release(); ❾
```

```
    }
  }

  private static String parseJoke(String json) { ❿
    int s = json.indexOf("\"value\":\"") + 9;
    int e = json.indexOf('"', s);
    return json.substring(s, e).replace("\\\"", "\"");
  }

  private static <T> T join(Future<T> f) {
    try {
      return f.get();
    } catch (InterruptedException e) { ⓫
      Thread.currentThread().interrupt();
      throw new CompletionException(e);
    } catch (ExecutionException e) {
      throw new CompletionException(e.getCause());
    }
  }
}
```

Let's go through what we did here:

- ❶ Configures a shared `HttpClient` with a 10-second connection timeout to prevent hanging on network issues.
- ❷ Defines the maximum number of concurrent API requests allowed at any time.
- ❸ Creates a counting semaphore that acts as our rate-limiting gate, initialized with the maximum parallel requests.
- ❹ Attempts to fetch 50 jokes, which exceeds our rate limit, demonstrating how the semaphore controls concurrency.
- ❺ Creates an executor that spawns a new virtual thread for each submitted task— perfect for I/O-bound operations.
- ❻ Blocks until all futures complete, collecting results in order of completion.
- ❼ Sets a per-request timeout of 30 seconds, preventing individual requests from blocking indefinitely.
- ❽ Acquires a permit from the semaphore, blocking if all permits are in use—this is where rate limiting happens.
- ❾ Releases the permit in a `finally` block, ensuring it's returned even if an exception occurs.
- ❿ Simple JSON parsing for demonstration—in production, use a proper JSON library like Jackson or Gson.
- ⓫ Improved error handling that preserves interruption status and unwraps `ExecutionException` to get the actual cause.

In this example, we've limited the concurrent tasks to 10 by setting the semaphore to count to 10. The task will be blocked whenever a new task tries to acquire the semaphore and the limit is reached. But remember, with virtual threads, blocking is cheap. This highlights the practical implications of virtual threads—even though we may introduce intentional bottlenecks for rate limiting, the efficiency of virtual threads minimizes the negative impact on the overall performance of the application.

Understanding Semaphores in Java

A *semaphore* is a synchronization mechanism that acts like a gatekeeper, controlling access to a shared resource. In Java, the `Semaphore` class (part of the java.util.concurrent package) offers a handy way to manage this access control.

The idea behind a semaphore is simple: it keeps track of a set number of permits. Before a thread can access a particular section of code, it needs to grab a permit. When the thread is finished, it returns the permit. If no permits are available, the thread waits until one is released.

Key methods of a semaphore are:

`acquire()`

This method requests a permit. The thread will wait if none are currently available.

`release()`

This method returns a permit to the semaphore, potentially allowing another waiting thread to proceed.

`availablePermits()`

This method lets you check how many permits are currently free.

Imagine you have a pool of only five database connections. You can use a semaphore to ensure that only five threads can use those connections at once. For instance:

```
import java.util.Optional;
import java.util.concurrent.Semaphore;

public class ResourcePool {
    private final Semaphore semaphore; ❶
    public ResourcePool(int resourceCount) {
        this.semaphore = new Semaphore(resourceCount); ❷
    }

    public Optional<String> useResource(String query) {
        try {
            semaphore.acquire(); ❸
            try {
                // Simulate obtaining and using a database connection
                return queryDatabase(query); ❹
            } finally {
                semaphore.release(); ❺
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); ❻
            return Optional.empty();
        }
    }

    private Optional<String> queryDatabase(String query) {
        // Simulate database query with some delay
        try {
            Thread.sleep(100);
```

```
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return Optional.empty();
            }
            return Optional.of("Result for: " + query);
        }
    }
```

Let's examine how this implementation controls access to our limited resources:

- ❶ The semaphore field stores our synchronization primitive. This semaphore acts as a thread-safe counter for available permits, managing concurrent access to our database connections.
- ❷ During construction, we set the resource limit. We initialize the semaphore with the maximum number of concurrent accesses allowed, effectively creating our connection pool size.
- ❸ This is where the actual rate limiting happens. The `acquire()` method blocks the current thread until a permit becomes available, implementing our resource constraint.
- ❹ Once a permit is acquired, we enter the critical section. This represents the section where the limited resource (database connection) is actually used.
- ❺ Resource cleanup is crucial in concurrent programming. We always release the permit in a `finally` block to prevent resource leaks, even if an exception occurs.
- ❻ Proper thread interruption handling is essential. We preserve the interrupted status for proper thread interruption handling, allowing callers to respond appropriately.

In the preceding class, the `useResource()` method is limited to access only by `resourceCount` number of threads. If more threads require access, they will have to wait until the threads with access release it.

In production systems, it's often valuable to monitor resource usage and implement timeouts to prevent indefinite blocking. Additionally, to gain an extra level of confidence in our semaphore's functionality, let's update our `ResourcePool` class.

Here's the revised code:

```
import java.util.Optional;
import java.util.Random;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class MonitoredResourcePool {
  private final Semaphore semaphore;
  private final AtomicInteger activeConnections; ❶
  private final AtomicInteger peakConnections; ❷

  public MonitoredResourcePool(int resourceCount) {
    this.semaphore = new Semaphore(resourceCount, true); ❸
```

```
    this.activeConnections = new AtomicInteger(0);
    this.peakConnections = new AtomicInteger(0);
  }

  public Optional<String> useResource(String query) {
    boolean acquired = false;
    try {
      acquired = semaphore.tryAcquire(5, TimeUnit.SECONDS); ❹
      if (!acquired) {
        return Optional.empty(); ❺
      }
      int current = activeConnections.incrementAndGet();
      peakConnections.updateAndGet(peak -> Math.max(peak, current)); ❻
      return queryDatabase(query);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      return Optional.empty();
    } finally {
      if (acquired) {
        activeConnections.decrementAndGet();
        semaphore.release();
      }
    }
  }

  public int getCurrentActiveConnections() {
    return activeConnections.get();
  }

  public int getPeakConnections() {
    return peakConnections.get(); ❼
  }

  private Optional<String> queryDatabase(String query) {
    try {
      Thread.sleep(new Random().nextInt(500) + 500);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      return Optional.empty();
    }
    return Optional.of("Result for: " + query);
  }
}
```

This enhanced implementation adds several production-ready features:

❶ Monitoring active connections helps with debugging and capacity planning. The `activeConnections` counter tracks current active connections for monitoring purposes only, not for enforcement of limits.

❷ Understanding peak usage patterns is crucial for capacity planning. This counter records the highest number of concurrent connections observed during the application's lifetime.

❸ Fairness prevents thread starvation in high-contention scenarios. By passing `true` as the second parameter, we create a fair semaphore that ensures threads acquire permits in FIFO order.

❹

Timeouts prevent indefinite blocking and improve system resilience. The `tryAcquire` method with timeout avoids indefinite blocking, returning control to the caller after the specified duration.

❺ Graceful degradation is better than hanging indefinitely. When unable to acquire a resource within the timeout period, we return an empty `Optional` to indicate failure.

❻ Atomic operations ensure thread-safe statistics collection. This thread-safe update of peak connections using atomic operations prevents race conditions in our monitoring code.

❼ Exposing metrics enables operational visibility. This method provides observability into resource pool usage patterns, which is essential for monitoring and alerting.

To verify that our semaphore correctly limits concurrent access, let's create a test that attempts to overwhelm the resource pool:

```
package ca.bazlur.modern.concurrency.c02;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.*;

public class ResourcePoolTest {

  public static void main(String[] args) throws Exception {
    int maxConcurrentThreads = 5;
    int totalRequests = 50;
    var pool = new MonitoredResourcePool(maxConcurrentThreads);

    var futures = new ArrayList<Future<Optional<String>>>();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
      for (int i = 0; i < totalRequests; i++) {
        final int taskId = i;
        futures.add(executor.submit(() -> pool.useResource("Query " + taskId)));
      }

      int successCount = 0;
      int timeoutCount = 0;

      for (Future<Optional<String>> future : futures) {
        Optional<String> result = future.get();
        if (result.isPresent()) {
          successCount++;
        } else {
          timeoutCount++; ❶
        }
      }

      System.out.printf("""
              requests  : %d
              successful: %d
              timed-out : %d
              peak usage: %d%n""",
          totalRequests, successCount, timeoutCount, pool.getPeakConnections());
```

```
            assert pool.getPeakConnections() <= maxConcurrentThreads
                : "Peak connections exceeded limit!";
        }
    }
}
```

Our test demonstrates the effectiveness of semaphore-based rate limiting:

1. Tracking timeouts helps us understand system behavior under load. This counter tracks requests that couldn't acquire a permit within the timeout period, indicating resource contention.
2. The peak connection count proves our rate limiting works correctly. This verification ensures the semaphore successfully limited concurrent access throughout the test execution.

When you run this test, you'll observe that despite submitting 50 concurrent requests, the peak connections never exceed our limit of 5. Some requests may timeout if they can't acquire a permit within five seconds, demonstrating the protective nature of our semaphore-based rate limiting.

The output appears as follows for me:

```
Total requests: 50
Successful: 32
Timed out: 18
Peak concurrent connections: 5
```

Why Use a Semaphore?

Having demonstrated how a semaphore effectively regulates concurrency, let's explore some common scenarios where this synchronization mechanism proves invaluable:

Resource management

: Semaphores are ideal for limiting access to shared resources such as network sockets, database connections, or any resource with a finite capacity. Consider a scenario where your application has a license for only 10 concurrent database connections. A semaphore initialized with 10 permits ensures you never exceed this limit, preventing license violations and connection errors.

Rate limiting

: Use semaphores to control the rate of requests to a web service or API, preventing overload and ensuring a smooth user experience. For example, if an external API allows only 100 requests per minute, we can use a semaphore in combination with a scheduled task that replenishes permits to maintain this rate limit.

General concurrency control

: Semaphores offer a way to precisely manage the number of threads simultaneously executing a specific code section. This is crucial for maintaining stability in highly concurrent applications,

especially when dealing with resources that can handle only a specific level of parallelism.

When using semaphores, keep the following important notes in mind:

Fairness

Java's `Semaphore` class allows you to configure fairness. In a fair semaphore, permits are granted in the order they were requested. However, fair semaphores can have slightly lower performance compared to nonfair ones due to the overhead of maintaining request order. Choose fairness when preventing thread starvation is more important than raw performance.

Blocking

The `acquire()` method can block a thread if no permits are available. Fortunately, the introduction of virtual threads in Java significantly reduces the overhead associated with this blocking. Unlike platform threads, blocked virtual threads don't consume OS resources, making semaphore-based synchronization much more scalable.

Error handling

Meticulously release permits within a `finally` block. This is essential to prevent resource leaks and ensure proper resource management, even in the event of exceptions.

Permit accounting

Remember that semaphores don't associate permits with specific threads. Any thread can release a permit, even one it didn't acquire. This flexibility can be useful but requires careful design to avoid accidentally increasing the effective permit count.

LIMITATIONS OF SEMAPHORES

Semaphores are essential for protecting limited resources, but they have a dangerous quirk: they don't track which thread acquired which permit. Any thread can release a permit, even one it never acquired.

Here's how easily things can go wrong:

```
// Start with 2 permits
Semaphore semaphore = new Semaphore(2);
// Thread 1: Forgets to release
Thread.ofVirtual().start(() -> {
    semaphore.acquire();
    // Oops! No release
});
// Thread 2: Releases without acquiring
Thread.ofVirtual().start(() -> {
    semaphore.release(); // BUG!
    semaphore.release(); // Double BUG!
});
// Result: We now have 3+ permits instead of 2!
```

In production, this could mean exceeding database connections, overwhelming APIs, or causing memory exhaustion.

Here's the bulletproof pattern:

```
public <T> T useResource(Callable<T> task)
  throws Exception {
    semaphore.acquire();  // Acquire before try
    try {
        return task.call();
    } finally {
        semaphore.release();  // ALWAYS releases
    }
}
```

In conclusion, understanding semaphores gives you a powerful tool for managing shared resources in your Java applications. This is particularly valuable when working with virtual threads, as it helps ensure stability and optimal performance.

Limitations of Virtual Threads

So far, we have learned the benefits of virtual threads. They are designed to be lightweight and easily scheduled by the Java runtime, offering a more efficient way to handle concurrency. However, they come with a limitation known as *pinning*, which could potentially affect the scalability and performance of your application.

In the context of virtual threads, pinning refers to the situation where a virtual thread becomes bound to its carrier thread (the underlying plat-

form thread on which it runs). While pinned, a virtual thread cannot unmount itself from the carrier thread even though it may hit blocking operations, effectively monopolizing that carrier thread for the duration of the pinning.

Pinning occurs in two main scenarios:

Synchronized blocks or methods

When a virtual thread enters a `synchronized` block or method, it becomes pinned to its carrier thread. This means that during the execution of that block or method, the carrier thread cannot be reused for other tasks.

Native methods or foreign functions

When a virtual thread executes a native method or a foreign function, it also becomes pinned.

The essence of virtual threads is their ability to be unmounted from carrier threads when they perform blocking operations, essentially freeing up the carrier threads for other tasks. When pinning happens, a virtual thread cannot unmount itself. This presents a challenge because we have limited carrier threads. If many virtual threads become pinned for extended periods, they can tie up these carrier threads. This blocks other virtual threads from executing, effectively limiting the concurrency benefits provided by virtual threads.

Pinning negates this advantage in the following ways:

Reduced throughput

Because a pinned virtual thread occupies its carrier thread, other virtual threads must wait for free carrier threads, reducing the system's overall throughput.

Resource inefficiency

Carrier threads are a finite resource tied to system capabilities. Having them blocked due to pinned virtual threads is an inefficient use of these resources.

Scalability concerns

If a significant portion of your virtual threads becomes pinned due to frequent use of `synchronized` blocks or native methods, you might run into scalability issues.

To alleviate the effects of pinning, consider the following strategies:

- Instead of `synchronized` blocks or methods, use `ReentrantLock` from java.util.concurrent.locks, as it allows the virtual thread to be unmounted when blocked.
- Regularly review your code to identify and minimize the use of synchronized methods or blocks and native methods in the context of virtual threads.

By understanding the limitations posed by pinning in virtual threads, you can make better architectural decisions for your Java applications.

Pinning

Let's take a look at a concrete Java example that demonstrates the concept of pinning in virtual threads:

```
import java.util.List;
import java.util.stream.IntStream;

public class ThreadPinnedExample {
  private static final Object lock = new Object();

  public static void main(String[] args) {
    List<Thread> threadList = IntStream.range(0, 10)
        .mapToObj(i -> Thread.ofVirtual().unstarted(() -> {
          if (i == 0) {
            System.out.println(Thread.currentThread()); ❶
          }
          synchronized (lock) { ❷
            try {
              Thread.sleep(25); ❸
            } catch (InterruptedException e) {
              Thread.currentThread().interrupt();
            }
          }
          if (i == 0) {
            System.out.println(Thread.currentThread()); ❹
          }
        })).toList();

    threadList.forEach(Thread::start);
    threadList.forEach(t -> {
      try {
        t.join();
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    });
  }
}
```

Let's trace through what happens when this code executes:

❶ For the first virtual thread (when `i == 0`), we print the current thread information before entering the `synchronized` block. This captures which carrier thread the virtual thread is running on.

❷ When a virtual thread enters this `synchronized` block, it becomes pinned to its carrier thread. The pinning occurs because `synchronized` blocks currently prevent virtual threads from unmounting from their carrier threads.

❸ The sleep operation simulates a blocking I/O operation. Normally, a sleeping virtual thread would unmount from its carrier thread, allowing that carrier to run other virtual threads. However, inside a `synchronized` block, the virtual thread remains pinned, monopolizing the carrier thread for the full 25 milliseconds.

❹ After exiting the `synchronized` block, we print the thread information again. This allows us to verify whether the virtual

thread remained on the same carrier thread.

When we execute this code, the output for thread `0` reveals the pinning effect:

```
VirtualThread[#21]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#21]/runnable@ForkJoinPool-1-worker-1
```

Notice that the carrier thread identifier (`ForkJoinPool-1-worker-1`) remains identical before and after the synchronized block. This confirms that the virtual thread did not switch carrier threads, indicating it was pinned for the duration of the blocking operation.

NOTE

Starting with JDK 24, virtual threads will no longer be pinned by `synchronized` blocks. The JVM now supports unmounting within synchronized sections, eliminating this limitation entirely. However, if you're using JDK 23 or earlier, you must still consider pinning behavior when designing concurrent applications.

Consider what happens if we scale this to 1,000 or 10,000 virtual threads, all trying to enter synchronized blocks with blocking operations. With only a handful of carrier threads available, most virtual threads will queue up waiting for a free carrier thread, defeating the purpose of using virtual threads for high concurrency.

Addressing the Pinning Problem with ReentrantLock

`ReentrantLock` is a synchronization mechanism in Java that offers more flexibility than the traditional `synchronized` block. It allows more sophisticated thread interactions and provides additional features such as fairness, `try` -lock, and interruptibility. One of the key advantages of using `ReentrantLock` is its ability to avoid the pinning problem in virtual threads.

The following example code showcases how to use `ReentrantLock` to avoid the pinning problem associated with virtual threads in Java. Instead of using a `synchronized` block, which would pin the virtual thread to its carrier, the code employs a `ReentrantLock` for synchronization. This allows the virtual thread to be unmounted from its carrier thread when it blocks, thus making the carrier thread available for other tasks. For example:

```
import java.util.concurrent.locks.ReentrantLock;
import java.util.stream.IntStream;

public class PreventPinningExample {
  private static final ReentrantLock lock = new ReentrantLock(); ❶

  public static void main(String[] args) {
    var threadList = IntStream.range(0, 10)
```

```
            .mapToObj(i -> Thread.ofVirtual().unstarted(() -> {
              if (i == 0) {
                System.out.println(Thread.currentThread()); ❷
              }
              lock.lock(); ❸
              try {
                Thread.sleep(25); ❹
              } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
              } finally {
                lock.unlock(); ❺
              }
              if (i == 0) {
                System.out.println(Thread.currentThread()); ❻
              }
            })).toList();

      threadList.forEach(Thread::start);
      threadList.forEach(thread -> {
        try {
          thread.join();
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
        }
      });
    }
}
```

Let's examine the key differences from our synchronized example:

❶ We create a `ReentrantLock` instance instead of using an object monitor. This lock provides the same mutual exclusion guarantees as a `synchronized` lock, but with virtual-thread-friendly behavior.

❷ We capture the initial carrier thread information for the first virtual thread.

❸ The `lock()` method acquires the lock, similar to entering a `synchronized` block. However, unlike `synchronized`, this doesn't pin the virtual thread.

❹ During the sleep operation inside the lock, the virtual thread can unmount from its carrier thread. The carrier becomes available to run other virtual threads while this one is blocked.

❺ The unlock operation must always occur in a `finally` block. This ensures the lock is released even if an exception occurs, preventing deadlocks.

❻ After releasing the lock, we print the thread information again to observe any carrier thread changes.

When you run this example, the output reveals the crucial difference:

```
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-3
```

The output can be interpreted as follows:

VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1

Indicates that the virtual thread with the ID # `20` is in a `runnable` state and is currently mounted on a carrier thread named `ForkJoinPool-1-worker-1`.

VirtualThread[#20]/runnable@ForkJoinPool-1-worker-3

Shows that the same virtual thread with the ID # `20` is still in a `runnable` state but has now been remounted onto a different carrier thread, named `ForkJoinPool-1-worker-3`.

This example shows that the virtual thread was initially running on one carrier thread and later moved to another. This is possible because we used `ReentrantLock` instead of a `synchronized` block. This essentially avoids the pinning problem and allows the Java runtime to optimally schedule the virtual thread on available carrier threads.

The key difference lies in implementation. While `synchronized` blocks use object monitors that currently require pinning, `ReentrantLock` uses park/unpark mechanisms that are virtual-thread-aware.

THE PARK/UNPARK MECHANISM

The park/unpark mechanism is a low-level thread coordination primitive in Java that forms the foundation for many higher-level concurrency utilities.

When a thread calls `LockSupport.park()`, it becomes "parked" (blocked) until:

- Another thread calls `unpark()` with this thread as the target.
- The thread is interrupted.
- The call spuriously returns (rare but possible).

Calling `LockSupport.unpark(thread)` makes the target thread eligible to proceed:

```
// Thread A
LockSupport.park(); // Blocks until unparked
// Thread B
LockSupport.unpark(threadA); // Wakes up Thread A
```

What makes park/unpark virtual-thread-friendly is that the JVM can detect when a virtual thread parks and unmount it from its carrier thread. This is different from object monitors (used by `synchronized`), which require the virtual thread to remain mounted.

When a virtual thread blocks on a `ReentrantLock`, the runtime can:

1. Save the virtual thread's state
2. Unmount it from the carrier thread
3. Use that carrier for other virtual threads

4. Later mount the virtual thread on any available carrier when the lock becomes available

---

SYNCHRONIZED BLOCKS AND VIRTUAL THREAD PINNING

While the `synchronized` keyword can lead to pinning, its impact varies depending on the specific code within the `synchronized` block. Let's consider a few examples:

EXAMPLE 1: MINIMAL PINNING RISK

```
synchronized (this) {
  return this.a + this.b;
}
```

In this case, the operation within the `synchronized` block is extremely short. Even if the virtual thread needs to wait for the lock, it will likely be unpinned very quickly, minimizing the impact.

EXAMPLE 2: POTENTIAL PINNING ISSUE

```
synchronized (this) {
  var response = httpClient.sendBlockingRequest(request);
  return response;
}
```

Here, the blocking network call (`httpClient.sendBlockingRequest`) within the `synchronized` block poses a pinning risk. The virtual thread could remain pinned for the entire duration of the network request, hindering concurrency gains.

EXAMPLE 3: SIMILAR PINNING ISSUE

```
synchronized (this) {
  this.wait();
}
```

Calls to `Object.wait()` also lead to pinning within `synchronized` blocks, as the virtual thread will remain pinned until notified.

KEY TAKEAWAY

The severity of pinning caused by `synchronized` depends on the nature of operations within the block. Short, non-blocking operations are generally fine. However, blocking calls necessitate careful consideration.

EVOLVING LANDSCAPE

Future JDK updates might introduce "unpinning-aware" I/O operations, potentially mitigating pinning in some scenarios. For now, when maximizing scalability with virtual threads, it's often wise to favor `ReentrantLock` for more flexible synchronization.

Special thanks to Simone Bordet for explaining this easily to me.

Native Method Invocation and Pinning

Similar to our previous experiments with `synchronized` blocks, let's explore how native method invocations can lead to virtual thread pinning. We'll demonstrate this by using a simple C function and Java's Foreign Function & Memory API, which is available as of JDK 22.

First, let's create a simple native function that simulates some work:

```
#include <unistd.h>
// Function definition
int addNumbers(int number1, int number2) {
    // Pause execution for 200,000 microseconds (200 milliseconds)
    usleep(200000);

    return number1 + number2;
}
```

Let's compile this function based on our operating system:

- Linux: `gcc -shared -fPIC -o libaddNumbers.so addNumbers.c`
- macOS: `gcc -shared -fPIC -o libaddNumbers.dylib addNumbers.c`
- Windows: Use a GCC environment (like MinGW) and `gcc -shared -o addNumbers.dll addNumbers.c`

After compiling on macOS, it provided me with `libaddNumbers.dylib`. Let's invoke this native method using virtual threads:

```
import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.IntStream;

public class ThreadPinnedNativeMethodExample {
    public static void main(String[] args) {
        List<Thread> threadList = IntStream.range(0, 10)
            .mapToObj(i -> Thread.ofVirtual().unstarted(() -> {
                if (i == 0) {
                    System.out.println(Thread.currentThread()); ❶
                }
                int sum = invokeNativeAddNumbers(56, 11); ❷
                if (i == 0) {
                    System.out.println(Thread.currentThread()); ❸
                }
            })).toList();
        threadList.forEach(Thread::start);
        threadList.forEach(t -> {
            try {
                t.join();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
```

```
    }

    public static int invokeNativeAddNumbers(int a, int b) {
        try (Arena arena = Arena.ofConfined()) { ❹
            SymbolLookup lookup = SymbolLookup.libraryLookup(
                Path.of("libaddNumbers.dylib"), arena); ❺
            MemorySegment memorySegment = lookup.find("addNumbers")
                .orElseThrow(() ->
                    new RuntimeException("addNumbers function not found"));
            Linker linker = Linker.nativeLinker();
            FunctionDescriptor addNumbersDescriptor = FunctionDescriptor.of(
                ValueLayout.JAVA_INT,     // return type
                ValueLayout.JAVA_INT,     // parameter 1
                ValueLayout.JAVA_INT);    // parameter 2
            MethodHandle addNumbersHandle = linker.downcallHandle(
                memorySegment, addNumbersDescriptor); ❻
            try {
                return (int) addNumbersHandle.invokeExact(a, b); ❼
            } catch (Throwable e) {
                throw new RuntimeException(e.getMessage());
            }
        }
    }
}
```

Let's understand what happens during execution:

❶ We capture the carrier thread information before the native call to establish a baseline.

❷ This is where the native method invocation occurs. During this call, the virtual thread becomes pinned to its carrier thread.

❸ After the native call completes, we check the carrier thread again to see if it changed.

❹ The Foreign Function & Memory (FFM) API is Java's modern approach to native interoperability, replacing the older Java Native Interface (JNI). Introduced as a preview feature in JDK 19 and finalized in JDK 22, it provides a safer, more performant way to call native code and manage off-heap memory. The `Arena` manages the lifecycle of native memory segments, ensuring proper cleanup when the `try`-with-resources block exits. To run this example with native access enabled, use: `java --enable-preview --source 21 --enable-native-access=ALL-UNNAMED ThreadPinnedNativeMethodExample.java`

❺ Adjust the library path based on your OS: use `.so` for Linux, `.dylib` for macOS, or `.dll` for Windows.

❻ The downcall handle creates a bridge between Java and the native function, allowing us to invoke C code from Java.

❼ During this native invocation, the virtual thread cannot unmount from its carrier thread, causing pinning for the entire 200ms duration.

The output should look like this:

```
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
```

```
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
```

The identical thread identifiers before and after calling `invokeNativeAddNumbers` confirm that the virtual thread was pinned while the native method was executing.

Now we can ask the question: Why did it really happen during the native method?

Native methods cause pinning because the JVM cannot inspect or control native code execution. When a virtual thread enters native code, several constraints come into play: native code might hold thread-local state that cannot be migrated between threads, the native call stack cannot be saved and restored like Java stack frames, and native code might interact directly with OS-level thread primitives. These limitations force the virtual thread to remain mounted on its carrier thread for the entire duration of the native call.

This pinning behavior means applications with frequent native calls may not fully benefit from virtual thread scalability. To mitigate this, consider batching operations to reduce native call frequency, using asynchronous native APIs where available, or reimplementing critical native functionality in pure Java. Always measure the actual impact of native calls on your application's concurrency to determine if they're becoming a bottleneck.

JEP 491: SYNCHRONIZE VIRTUAL THREADS WITHOUT PINNING

JEP 491: "Synchronize Virtual Threads Without Pinning", delivered in JDK 24, takes a significant step forward by reworking the synchronized keyword to be more virtual-thread-friendly. In its current state, when a virtual thread enters a synchronized block, it gets pinned to a platform thread. With this update, virtual threads will be able to acquire, hold, and release monitors independently of their carrier threads. The JVM scheduler will now allow blocked virtual threads to unmount from platform threads, freeing up resources so other tasks can continue running efficiently.

That being said, pinning isn't completely eliminated. Some edge cases such as blocking inside class initializers, waiting on class initialization by another thread, or resolving symbolic references during class loading or native code calls, will still result in pinning. While these situations are relatively rare, they could pose issues in highly concurrent applications. JEP 491 proposes monitoring these cases and refining the approach in future updates if needed.

However, there's a catch: to take advantage of these improvements, you'll need to migrate to JDK 24+. Since JDK 21 is still the long-term support (LTS) version that most applications rely on, it makes sense to remain aware of pinning issues and design applications accordingly.

For example, if we run the following code using JDK 25, we will not see the pinning issue anymore:

```
java --enable-native-access=ALL-UNNAMED ThreadPinnedNativeMethodExample.java
```

The output looks like this:

```
VirtualThread[#26]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#26]/runnable@ForkJoinPool-1-worker-6
```

The Conundrum of ThreadLocal Variables in Virtual Threads

`ThreadLocal` variables in Java offer a way to confine data to a specific thread. The `ThreadLocal` class lets you create variables that are readable and writable only by the thread that created them. This eliminates the need for synchronization in scenarios where multiple threads might try to access the same data simultaneously.

Some classic use cases are:

Resource isolation

> `ThreadLocal` variables are perfect for storing resources that aren't thread-safe. A classic example is `SimpleDateFormat`,[1] where each thread can have its own instance to avoid conflicts.

Implicit context

They're also commonly used to store contextual information specific to a thread's task, such as a database connection, user session data, or transaction IDs.

Challenges with Virtual Threads

Virtual threads in Java are designed to be lightweight, allowing you to potentially run millions within a single application. This massive scale creates challenges when overusing `ThreadLocal` variables:

Memory consumption

Each virtual thread having its own copy of a `ThreadLocal` variable can rapidly increase memory usage, especially if the stored data is large.

Overhead

Initializing and cleaning up `ThreadLocal` variables comes with overhead. With the massive number of virtual threads, these actions can add a performance burden.

Inheritance:

Virtual threads inherit `ThreadLocal` values from their parent thread like traditional threads. This inheritance can introduce subtle bugs that are difficult to trace and debug.

Let's examine a code example that demonstrates the potential drawbacks of heavily relying on `ThreadLocal` variables with virtual threads:

```
import java.time.Duration;
import java.util.stream.IntStream;

public class ThreadLocalExample {

    public static void main(String[] args) {
        ThreadLocal<LargeObject> threadLocal
                = ThreadLocal.withInitial(LargeObject::new);

        var threadList = IntStream.range(0, 1000)
                .mapToObj(i -> Thread.ofVirtual().unstarted(() -> {
                    LargeObject largeObject = threadLocal.get();
                    useIt(largeObject);
                    sleep();
                })).toList();

        threadList.forEach(Thread::start);
        threadList.forEach(thread -> {
            try {
                thread.join();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
    }

    private static void useIt(LargeObject largeObject) {
        System.out.println(largeObject.data.length);
```

```
        }

        private static void sleep() {
            try {
                Thread.sleep(Duration.ofMinutes(5));
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }

        static class LargeObject {
            private byte[] data = new byte[1024 * 500]; // 500 KB
        }
    }
```

This example demonstrates the creation of a 500 KB `LargeObject`. Each of the 1,000 virtual threads stores its own copy of a `LargeObject` within a `ThreadLocal`. By opening JConsole[2] after executing the program, we can observe the memory usage (Figure 2-3).

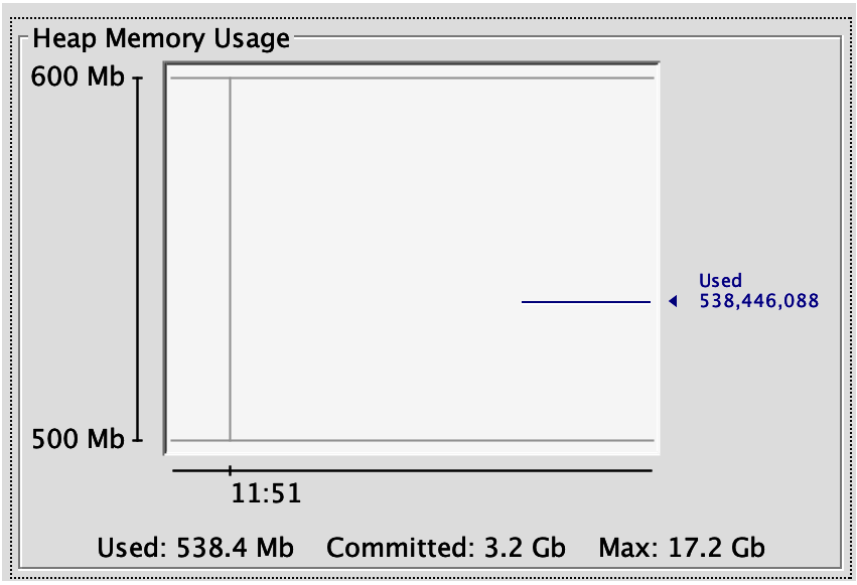

Figure 2-3. Heap memory usage on JConsole when using `ThreadLocal`

It is evident from this observation that memory usage has accumulated. However, if we execute the same program without using `ThreadLocal`, the memory usage drops immediately, despite the involvement of 1,000 virtual threads (Figure 2-4).

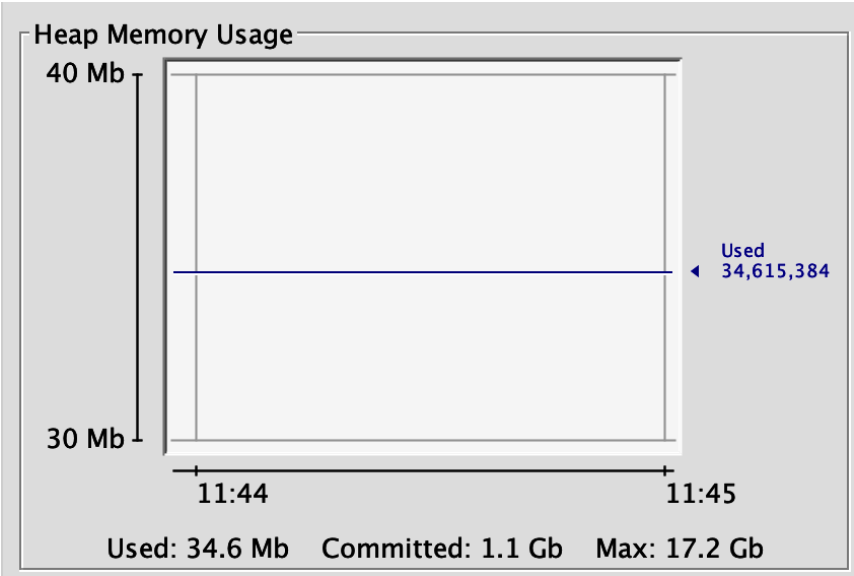

Figure 2-4. Heap memory usage on JConsole when not using `ThreadLocal`

The memory and overhead challenges associated with `ThreadLocal` in a virtual thread environment motivate the exploration of alternatives. Here are two key strategies:

Scoped values

Scoped values are designed with virtual threads in mind. Their immutability and bounded lifetimes make them well-suited for passing data between threads in a safe and efficient manner. We'll cover scoped values in depth in Chapter 5.

Rethinking sharing

The advent of virtual threads prompts us to re-evaluate our overall approach to data sharing. Explore if it's possible to restructure your application to minimize the need for `ThreadLocal` altogether, potentially leading to more scalable designs.

Monitoring

Now that we understand the limitations of virtual threads, it's clear they arise mainly from two sources: pinning and the use of `ThreadLocal`. Crafting new applications with these limitations in mind is one thing, but we don't always have that luxury. Often, we're working with legacy applications, some boasting millions of lines of code. Migrating to virtual threads in such cases requires a keen eye for compatibility issues. Fortunately, we're not left to our own devices here; there are tools specifically designed to flag potential problems.

Monitoring ThreadLocals

To track the usage of `ThreadLocal`, you can start the JVM with the `-Djdk.traceVirtualThreadLocals` flag. This will print stack traces whenever a `ThreadLocal` variable is accessed within a virtual thread, highlighting potential mismanagement or overuse.

We can use the following command to monitor `ThreadLocal` activity in our `ThreadLocalExample`:

```
java -Djdk.traceVirtualThreadLocals ThreadLocalExample.java
```

Executing this will yield an output like the following:

```
VirtualThread[#23]/runnable@ForkJoinPool-1-worker-4
...
VirtualThread[#25]/runnable@ForkJoinPool-1-worker-6
    java.base/java.lang.ThreadLocal.setInitialValue(ThreadLocal.java:236)
    java.base/java.lang.ThreadLocal.get(ThreadLocal.java:194)
    java.base/java.lang.ThreadLocal.get(ThreadLocal.java:172)
    com.example.myapp.ThreadLocalExample.
                        lambda$main$0(ThreadLocalExample.java:13)
    java.base/java.lang.VirtualThread.run(VirtualThread.java:329)
```

Each entry provides valuable insight into where and how `ThreadLocal` is used, allowing us to refine our approach to ensure compatibility with virtual threads.

Upon examination of the output, several patterns emerge. Initially, we see the `ThreadLocal.setInitialValue()` being invoked, an anticipated action since each virtual thread must initialize its thread-local variable. The stack traces with `java.base/java.lang.ThreadLocal.setInitialValue(ThreadLocal.java:236)` confirm this.

Furthermore, when a thread needs to retrieve its thread-local variable, it calls `ThreadLocal.get()`. The output, marked by instances of `java.base/java.lang.ThreadLocal.get(ThreadLocal.java:194)`, underscores these moments of access.

Most importantly, the stack trace sheds light on the specific locations within your code where `ThreadLocal` is accessed. The lines with `com.example.myapp.ThreadLocalExample.lambda$main$0(ThreadLocalExample.java:13)` point directly to these critical points in your application, giving you the exact references needed for further investigation or refactoring.

As it becomes apparent that every virtual thread initializes its own `ThreadLocal` instance, it reconfirms our concern: the potential for escalated memory use, particularly when virtual threads are many in number. This aspect requires careful consideration when migrating to virtual threads.

Simultaneously, the stack trace gives us invaluable information for debugging. It provides a window into precisely where `ThreadLocal` is being used. With this, we can enhance the performance of our application, ensuring that `ThreadLocal` is not only used judiciously but also replaced or restructured if it becomes a bottleneck.

Monitoring Pinning

Similar to our approach with `ThreadLocal`, we can monitor existing source code for pinning issues. We have multiple tools available at our disposal. These tools range from JVM flags to the Java Flight Recorder (JFR) and `jcmd` for thread dumps. Let's begin by examining the utility of the JVM flag.

Using the JVM flag

The system property `jdk.tracePinnedThreads` is designed to trigger a stack trace whenever a thread encounters a blocking operation while pinned. When the JVM is run with `-Djdk.tracePinnedThreads=full`, it prints a complete stack trace that includes not just the Java frames but also the native frames and those frames holding monitors, which are highlighted. For a more succinct output that focuses on the core issue, `-Djdk.tracePinnedThreads=short` will limit the trace to just the problematic frames.

To illustrate this, let's execute the `ThreadPinnedExample.java` with the JVM flag:

```
java -Djdk.tracePinnedThreads=full ThreadPinnedExample.java
```

Here's a snippet of what the output might look like:

```
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#21]/runnable@ForkJoinPool-1-worker-2 reason:MONITOR
    java.base/java.lang.VirtualThread$VThreadContinuation.onPinned(VirtualThread
    .java:199)
    java.base/jdk.internal.vm.Continuation.onPinned0(Continuation.java:393)
    java.base/java.lang.VirtualThread.parkNanos(VirtualThread.java:635)
    java.base/java.lang.VirtualThread.sleepNanos(VirtualThread.java:812)
    java.base/java.lang.Thread.sleepNanos(Thread.java:489)
    java.base/java.lang.Thread.sleep(Thread.java:522)
    ca.example.myapp.
.ThreadPinnedExample.lambda$main$0(ThreadPinnedExample.java:20) <== monitors:1
    java.base/java.lang.VirtualThread.run(VirtualThread.java:329)
VirtualThread[#20]/runnable@ForkJoinPool-1-worker-1
```

This output documents each instance where a thread becomes pinned. In particular, the `reason:MONITOR` tag indicates a thread is waiting to acquire a monitor, which can be an invaluable clue in detecting potential bottlenecks. The stack trace not only provides the *where* but also the *why* —in this case, highlighted by the line ending with `<== monitors:1`, denoting a thread holding a monitor.

By dissecting this output, we can make informed decisions about optimizing thread usage and minimizing pinning by identifying the affected area of code and potentially replacing it.

Using the Java Flight Recorder

Java Flight Recorder (JFR) is an invaluable tool that extends beyond traditional monitoring, offering specialized capabilities for observing and debugging the nuances of virtual threads. This powerful feature set facilitates a thorough and effective debugging process, which is vital as we navigate the complexities of concurrent programming.

JFR provides powerful tools to monitor virtual thread behavior. Here's a look at the most important events:

- `jdk.VirtualThreadStart` and `jdk.VirtualThreadEnd` mark the birth and termination of a virtual thread. While not enabled by default, turning them on gives you detailed tracking of thread lifecycles, which can be helpful for debugging or fine-grained analysis.
- `jdk.VirtualThreadPinned` is a crucial event, enabled by default, that signals when a virtual thread becomes pinned to a carrier thread for an extended period (the threshold is configurable, 20ms being the default). Pinning events indicate potential performance bottlenecks where the efficiency gains of virtual threads are lost.
- `jdk.VirtualThreadSubmitFailed` is another default-enabled event, and it signals failures to start or unpark virtual threads. It can point to resource exhaustion or unexpected bottlenecks within the JVM's thread management.

To monitor specific events like `jdk.VirtualThreadStart` and `jdk.VirtualThreadEnd`, we have two main approaches: utilizing JDK Mission Control for a GUI-driven experience or creating a custom JFR configuration file for more direct control.

Creating a custom JFR configuration file (*.jfc*) is a straightforward way to track the events we're most interested in. This tailored file enables selective monitoring, focusing your resources and analyzing relevant data points.

Here's a basic template for a *.jfc* file designed to record pivotal virtual thread events:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration version="2.0" label="Virtual Thread Events"
      description="JFR configuration to record virtual thread events">
  <event name="jdk.VirtualThreadStart">
  <setting name="enabled">true</setting>
  </event>
  <event name="jdk.VirtualThreadEnd">
  <setting name="enabled">true</setting>
  </event>
  <event name="jdk.VirtualThreadPinned">
  <setting name="enabled">true</setting>
  </event>
  <event name="jdk.VirtualThreadSubmitFailed">
  <setting name="enabled">true</setting>
  </event>
</configuration>
```

To put this into action, let's put the XML content into a file named *VThreadEvents.jfc*.

To understand how to monitor virtual threads effectively, let's dive into a code example designed to showcase JFR in action. Here's the Java program, and we'll break it down afterward:

```
import java.time.Duration;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class JFRVirtualThreadDemo {
  private static final Object syncLock = new Object();
  private static final Lock reentrantLock = new ReentrantLock();
  public static void main(String[] args) {
    // Triggering lifecycle events for virtual threads
    Thread vThreadStartEnd = Thread.ofVirtual().unstarted(() -> {
      System.out.println("Virtual thread started and will end soon.");
    }); ❶
    vThreadStartEnd.start();
    joinThread(vThreadStartEnd);
    // Pinning with a synchronized block
    Thread vThreadPinnedSync = Thread.ofVirtual().unstarted(() -> {
      synchronized (syncLock) { ❷
        sleepUninterruptibly(Duration.ofMillis(500));
      }
    });
    vThreadPinnedSync.start();
    joinThread(vThreadPinnedSync);
    // No pinning with ReentrantLock
    Thread vThreadWithLock = Thread.ofVirtual().unstarted(() -> {
      reentrantLock.lock();
      try {
        sleepUninterruptibly(Duration.ofMillis(500)); ❸
      } finally {
        reentrantLock.unlock();
      }
    });
    vThreadWithLock.start();
    joinThread(vThreadWithLock);
  }

  private static void joinThread(Thread thread) {
    try {
      thread.join();
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
  }

  private static void sleepUninterruptibly(Duration duration) { ❹
    boolean interrupted = false;
    try {
      long remainingNanos = duration.toNanos();
      long end = System.nanoTime() + remainingNanos;
      while (true) {
        try {
          Thread.sleep(remainingNanos / 1_000_000,
              (int) (remainingNanos % 1_000_000));
```

```
            return;
          } catch (InterruptedException e) {
            interrupted = true;
            remainingNanos = end - System.nanoTime();
          }
        }
      } finally {
        if (interrupted) {
          Thread.currentThread().interrupt();
        }
      }
    }
}
```

Let's examine what each section demonstrates:

1. This simple virtual thread showcases basic lifecycle events. JFR will record both `VirtualThreadStart` and `VirtualThreadEnd` events, showing how quickly virtual threads complete short tasks.
2. The `synchronized` block causes thread pinning. When the virtual thread sleeps inside this block, it remains pinned to its carrier thread for the entire 500ms duration, which JFR will capture as a `VirtualThreadPinned` event.
3. Despite using a lock, `ReentrantLock` doesn't cause pinning. The virtual thread can unmount from its carrier during sleep, demonstrating why `ReentrantLock` is preferred for virtual threads.
4. This helper method ensures the thread sleeps for the full duration, even if interrupted, making our timing measurements more predictable for demonstration purposes.

Now that we've prepared everything, it's time to start a recording and execute our example code. Here's the command to initiate the JFR recording:

```
java -XX:StartFlightRecording=filename=recording.jfr,settings=VThreadEvents.jfc \
JFRVirtualThreadDemo.java
```

This will record only the events we've specified in our custom *.jfc* file.

After running our application, we can analyze the *recording.jfr* file using tools like JDK Mission Control, or we can print the events to the console using:

```
jfr print --events jdk.VirtualThreadStart,jdk.VirtualThreadEnd,\
jdk.VirtualThreadPinned,jdk.VirtualThreadSubmitFailed recording.jfr
```

This way, we'll get insights specifically into the virtual thread behavior of our Java application:

```
jdk.VirtualThreadStart {
  startTime = 10:23:14.936 (2024-03-23)
  javaThreadId = 23
```

```
    eventThread = "" (javaThreadId = 23, virtual)
  }
  jdk.VirtualThreadEnd {
    startTime = 10:23:14.939 (2024-03-23)
    javaThreadId = 23
    eventThread = "" (javaThreadId = 23, virtual)
  }
  jdk.VirtualThreadStart {
    startTime = 10:23:14.940 (2024-03-23)
    javaThreadId = 35
    eventThread = "" (javaThreadId = 35, virtual)
  }
  jdk.VirtualThreadPinned {
    startTime = 10:23:14.941 (2024-03-23)
    duration = 504 ms
    eventThread = "" (javaThreadId = 35, virtual)
  }
  jdk.VirtualThreadEnd {
    startTime = 10:23:15.445 (2024-03-23)
    javaThreadId = 35
    eventThread = "" (javaThreadId = 35, virtual)
  }
  jdk.VirtualThreadStart {
    startTime = 10:23:15.446 (2024-03-23)
    javaThreadId = 37
    eventThread = "" (javaThreadId = 37, virtual)
  }
  jdk.VirtualThreadEnd {
    startTime = 10:23:15.948 (2024-03-23)
    javaThreadId = 37
    eventThread = "" (javaThreadId = 37, virtual)
  }
```

These logs show how quickly virtual threads are created and finished, highlighting how they're meant for short tasks. Additionally, the logs reveal when virtual threads are pinned. Pinning times tell us how long a virtual thread remains attached to a carrier thread, which can help identify potential slowdowns.

Additionally, we can load the *recording.jfr* file to JDK Mission Control or Azul Mission Control and analyze it (Figure 2-5).

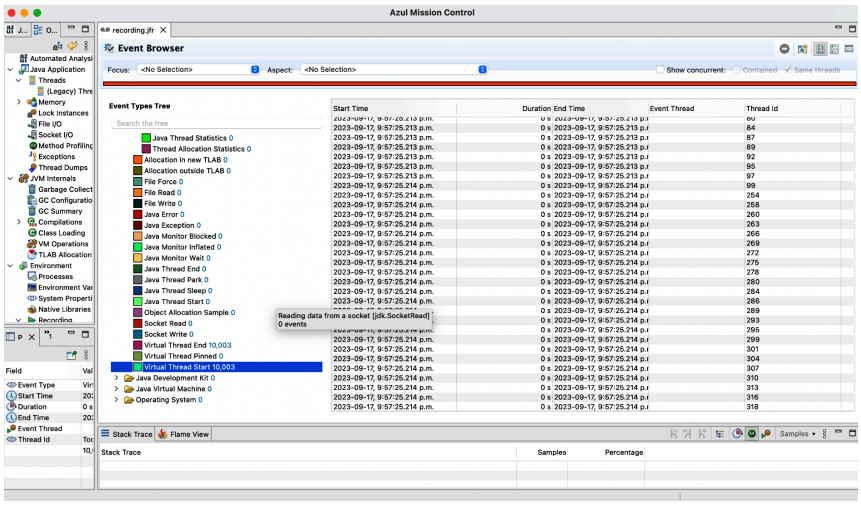

Figure 2-5. Java Flight Recorder analysis using Azul Mission Control

Viewing Virtual Threads in jcmd Thread Dumps

While tools like JDK Mission Control offer sophisticated analysis, sometimes a quick thread dump is a useful diagnostic tool. The `jcmd` utility provides this functionality, and importantly, it lets us capture information about virtual threads.

To generate thread dumps, first obtain the process ID (PID) of your running Java application using tools like `ps` or `jps`.[3] Then, choose your desired format for the thread dump.

Plain text

```
jcmd <PID> Thread.dump_to_file -format=text <file>
```

JSON

```
jcmd <PID> Thread.dump_to_file -format=json <file>
```

Replace `<PID>` with the actual process ID and `<file>` with the desired output filename.

For example, you can execute the following command to generate a JSON thread dump:

```
jcmd 12345 Thread.dump_to_file -format=json threaddump.json
```

The JSON format is particularly useful for automated debugging and analysis tools that can consume JSON data. It's a machine-readable format that allows you to programmatically analyze thread states, which can be beneficial in complex systems with many threads.

The `jcmd` thread dump will list virtual threads that are blocked in network I/O operations and virtual threads that are created by the `ExecutorService` interface. However, it has limitations as it will not include:

- Object addresses
- Locks
- Java Native Interface (JNI) statistics
- Heap statistics
- Other information commonly found in traditional thread dumps

This focus on essential information makes it easier to pinpoint issues specifically related to thread execution and blocking, without the noise of additional, often irrelevant, details.

By using `jcmd` along with JFR, we can get a comprehensive view of how virtual threads are performing in our application, enabling us to debug and optimize more effectively.

Programmatic thread dumps in Java

In addition to using the command-line utility, you can also generate thread dumps programmatically within your Java application. The fol-

lowing Java code snippet demonstrates this technique:

```
import java.io.IOException;
import java.lang.ProcessHandle;
import java.time.Duration;
import java.util.List;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.stream.IntStream;

public class ThreadDumpDemo {
  private static final int THREAD_COUNT = 1_000;
  private static final Duration WORK_DURATION = Duration.ofSeconds(5);
  private static final Duration DELAY_BEFORE_DUMP = Duration.ofSeconds(2);

  public static void main(String[] args) {
    long pid = ProcessHandle.current().pid(); ❶
    String outputFile = "dump.json";
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
      IntStream.range(0, THREAD_COUNT).forEach(i ->
          executor.submit(() -> sleep(WORK_DURATION))); ❷
      executor.submit(() -> {
        sleep(DELAY_BEFORE_DUMP); ❸
        runJcmdDump(pid, outputFile);
      });
    }
  }

  private static void sleep(Duration d) {
    try {
      TimeUnit.NANOSECONDS.sleep(d.toNanos());
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
  }

  private static void runJcmdDump(long pid, String file) {
   ProcessBuilder pb = new ProcessBuilder(List.of(
    "/bin/sh", "-c",
    String.format("jcmd %d Thread.dump_to_file -format=json %s",
        pid, file))); ❹
    try {
      Process p = pb.start();
      int exit = p.waitFor();
      if (exit != 0) {
        System.err.printf("jcmd exited %d%n", exit);
        p.getInputStream().transferTo(System.err);
        p.getErrorStream().transferTo(System.err);
      }
    } catch (IOException | InterruptedException e) {
      Thread.currentThread().interrupt();
      System.err.println("Failed to run jcmd: " + e.getMessage());
    }
  }
}
```

Let’s understand how this program captures thread state:

❶

We obtain the current PID using the `ProcessHandle` API. This PID is essential for the `jcmd` tool to identify which JVM process to dump.

❷ We create 1,000 virtual threads, each sleeping for 5 seconds. This simulates a heavily concurrent workload, ensuring that many active threads are present when the dump is obtained.

❸ The thread dump is triggered after a two-second delay. This timing ensures that most virtual threads are still active (or sleeping) when we capture the system state.

❹ We use `ProcessBuilder` to execute the `jcmd` command with JSON output format. The JSON format provides structured data that's easier to parse programmatically than traditional text dumps.

This program demonstrates how to programmatically trigger and capture thread dumps using Java's `ProcessBuilder`. It schedules a large number of virtual threads and then generates a thread dump when the last thread is about to be scheduled. The dump is saved to a file, and the PID is printed to the console for reference.

The generated *dump.json* file provides rich information about the system state:


```
{
  "threadDump": {
    "processId": "76586",
    "time": "2024-03-23T15:01:41.901030Z",
    "runtimeVersion": "21+35-2513",
    "threadContainers": [
      {
        "container": "<root>",
        "parent": null,
        "owner": null,
        "threads": [
         {
           "tid": "1",
           "name": "main",
           "stack": [
              "java.base\/java.io.FileInputStream.readBytes(Native Method)",
              "java.base\/java.io.FileInputStream.read(FileInputStream.java:287)"
              "java.base\/java.io.BufferedInputStream.read1(BufferedInputStream.j
              "java.base\/java.io.BufferedInputStream.implRead(BufferedInputStrea
               ... (additional stack trace elements)"
           ]
         },
         ....
         "... (additional threads)"
        ],
        "threadCount": "7"
      },
      {
        "container": "java.util.concurrent.ThreadPerTaskExecutor@768b771c",
        "parent": "<root>",
        "owner": null,
        "threads": [
         {
           "tid": "20",
```

```
                "name": "",
                "stack": [
                    "java.base\/java.lang.VirtualThread.parkNanos(VirtualThread.java:63
                    "java.base\/java.lang.VirtualThread.sleepNanos(VirtualThread.java:8
                    "java.base\/java.lang.Thread.sleep(Thread.java:507)",
                    "ThreadDumpDemo.sleepOfSeconds(ThreadDumpDemo.java:38)",
                    "ThreadDumpDemo.lambda$main$0(ThreadDumpDemo.java:29)",
                    "java.base\/java.util.concurrent.ThreadPerTaskExecutor$TaskRunner
                        .run(ThreadPerTaskExecutor.java:314)",
                    "java.base\/java.lang.VirtualThread.run(VirtualThread.java:311)"
                ]
            }
            "... (additional threads)"
        "... (additional thread containers)"
    ]
  }
}
```


Now let's see what it reveals:

Main thread

The thread with `tid: "1"` named `main` indicates the main thread. This is where our application starts. It's currently doing a file input operation.

System threads

Threads like `Reference Handler`, `Finalizer`, `Signal Dispatcher`, and `Notification Thread` are JVM-managed threads that handle garbage collection, reference objects, and other JVM-level tasks.

Common-cleaner thread

This thread is responsible for cleaning the reference objects after they are processed by the garbage collector.

Many virtual threads

We see lots of threads that are likely virtual threads. These are the numerous unnamed threads with `tid`s such as `20`, `22`, `23`, etc. They are virtual threads, given their stack traces include `java.lang.VirtualThread.parkNanos` and `java.lang.VirtualThread.sleepNanos`, likely due to our `sleepOfSeconds` calls.

In a real troubleshooting scenario, you'd look for these clues in a thread dump:

Blocked threads

Threads stuck in the `BLOCKED` or `WAITING` state often indicate problems like deadlocks or resource contention.

Resource usage

Combine the thread dump with CPU and memory usage data to see if the system is overloaded, which can also create bottlenecks.

Generating Thread Dumps with HotSpotDiagnosticsMXBean

Java Management Extensions (JMX) offer powerful tools for monitoring applications. The `HotSpotDiagnosticMXBean` within the com.sun.management package has been enhanced with a new method that lets you specify the desired format of your thread dumps. This includes JSON for structured analysis, or the traditional plain-text format.

Here's how to generate a JSON thread dump programmatically:

```
public static void takeThreadDump(String outputFile) {
  var hotSpotDiagnosticMXBean
      = ManagementFactory.getPlatformMXBean(HotSpotDiagnosticMXBean.class);
  try {
    // Ensure that the output file path is absolute
    if (!new File(outputFile).isAbsolute()) {
      throw new IllegalArgumentException("Output path must be absolute.");
    }
    hotSpotDiagnosticMXBean.dumpThreads(outputFile,
        HotSpotDiagnosticMXBean.ThreadDumpFormat.JSON);
  } catch (IOException e) {
    throw new RuntimeException("An error occurred while taking thread dump", e);
  }
}
```

The `dumpThreads` is a new method that has been added which takes an argument from the `ThreadDumpFormat` enum, allowing you to choose between `TEXT_PLAIN` or `JSON`. Also, you need to make sure to always provide an absolute path for the `outputFile` to avoid errors. This method can be used with JMX tools for both local and remote diagnostics.

Practical Tips for Migrating to Virtual Threads

Now that we have discussed the benefits and limitations of virtual threads and how to debug them, here are some tips when migrating:

Library updates are key

: The best way to avoid pinning issues is to use libraries specifically updated for virtual threads. These libraries will use modern synchronization tools that avoid tying up carrier threads.

When updates aren't available

: If we can't update libraries, we can move legacy I/O and other blocking operations to traditional thread pools (`Executors.newFixedThreadPool()`). This keeps blocking code contained.

The semaphore's dilemma

: Semaphores can limit how many virtual threads enter a pinning-prone code section. However, use them very carefully. Setting the

limit too low will choke off your application's concurrency and negate the benefits of virtual threads.

While the advent of virtual threads brings significant benefits, it's important to consider several factors to leverage their potential and fully avoid common pitfalls:

Ecosystem evolution

Many libraries are still adapting to virtual threads. We need to be patient and check for updates regularly.

Resource trade-offs

Virtual threads let us run more tasks concurrently, but this could lead to other resource bottlenecks. We may still need ways to limit usage.

Know your framework

We need to research how our preferred frameworks and languages work with virtual threads. There might be specific patterns to avoid.

As we adopt virtual threads, we need to up our game on monitoring. Focus on these key metrics:

CPU

We want to see those scalability gains! Track CPU usage patterns to ensure you're efficiently using virtual threads and minimizing unexpected pinning events.

Memory and garbage collection

Virtual threads shift memory usage patterns. We may need to adjust our heap sizes or fine-tune garbage collection if there are changes in behavior.

Latency and throughput

This is the ultimate test. Are our users experiencing faster response times? Is our system handling a higher volume of requests under load? These metrics will quantify the real-world impact of our migration.

Reaffirming the Benefits of Virtual Threads

We've explored the mechanics of virtual threads, monitoring techniques, and potential areas for caution; let's revisit the core advantages that make them a compelling addition to Java's concurrency toolkit:

Simplified concurrency

Virtual threads fundamentally streamline the way we write concurrent code. Their lightweight nature allows us to express tasks and workflows more naturally, reducing cognitive overhead and the potential for errors compared to traditional thread management.

Enhanced scalability

Applications using virtual threads can often handle far greater levels of concurrency without hitting resource limits imposed by

operating system threads. This scalability is especially valuable for modern applications handling numerous simultaneous connections or requests.

Resource efficiency

Virtual threads consume minimal memory and are scheduled intelligently by the JVM. This translates directly into better hardware utilization, potentially reducing operational costs or enabling you to do more with the same hardware resources.

Compatibility

A significant advantage of virtual threads is their integration with existing Java paradigms. You can often introduce them into your applications to reap performance benefits without major code refactoring.

Reduced CPU overhead

Because virtual threads don't directly consume CPU time when idle (e.g., waiting for I/O), they allow your application to dedicate more processing power to active tasks, improving overall responsiveness.

Streamlined server-side programming

Virtual threads promise to revitalize the classic "request-per-thread" model for server-side applications, making it far more scalable than when using traditional threads.

It's About Scalability

The notion of scalability in Java is all about the way an application handles an increase in workload. Traditionally, the way threads interact with I/O operations has been the great bottleneck to scalability. We can think of platform threads as being like bank tellers where people are waiting in line, but they don't do anything while they're waiting for information to process or for a network reply.

This is where virtual threads totally change the game. They're so lightweight that you can have thousands or even millions of them running in parallel without stressing your system. What's more, during I/O waits, virtual threads can gracefully "unmount" from their underlying carrier threads, freeing up valuable memory in the process. This means your system can juggle far more tasks at the same time.

With virtual threads, we can finally unleash the full power of modern CPUs. Their ability to support a massive number of virtual threads helps us squeeze every bit of performance out of our hardware. More threads translate into higher throughput and a more responsive application, even when it's being hammered with requests.

The benefits for your scalable bank (application) are:

Serving more customers (concurrent requests)

Now your bank—or rather, your application—can handle a much larger volume of customer requests without getting bogged down.

Reduced wait times (improved throughput)

This efficient use of resources means faster overall service, with more customers being helped in the same amount of time.

Optimized resource utilization (lower memory footprint)

Virtual threads are lean so that you can serve more customers with the same number of tellers (underlying resources).

Virtual threads represent a huge leap for scalable Java applications. By breaking free from the constraints of traditional threads, they pave the way for systems that are more efficient, responsive, and capable of handling ever-growing demands.

In Closing

The introduction of virtual threads in Java represents a significant shift in the history of the Java concurrency model, significantly increasing scalability and performance for applications run on services. As you begin utilizing virtual threads in your applications, exploit their ability to handle a vast number of concurrent tasks with minimal resource overhead. This can lead to more performant, responsive, and scalable applications.

Start by introducing virtual threads in parts of your application use cases likely to benefit the most, such as I/O-bound operations. Gradually ramp up their usage while closely monitoring the impact on performance and resource usage.

Keep a vigilant eye on thread activity, identify potential bottlenecks, and ensure you are fully leveraging the advantages of virtual threads. Remember the limitations of pinning and `ThreadLocal`. Consider employing modern synchronization tools like `ReentrantLock` to avoid unnecessary pinning. Stay informed about the latest library and framework support to take advantage of optimizations and new features.

Reevaluate your concurrency patterns and fully embrace virtual threads' simpler and more natural concurrency models. By embracing virtual threads, you can simplify and make your concurrency more coherent, significantly improving your applications' performance and scalability, resulting in cleaner and easier-to-maintain code.

As Java continues to evolve, staying informed and adaptable will ensure that you can fully leverage these powerful new capabilities, unlocking the full potential of your applications.

[1] Java now offers the `DateTimeFormatter` class, which is thread-safe and eliminates the need for `ThreadLocal` in this specific case.

[2] JConsole is a graphical monitoring tool bundled with the Java Development Kit (JDK).

[3] We can identify a process's PID by using the `ps` command for general processes or `jps` specifically for Java applications. Use `ps -ef | grep <process_name>` to list all processes and filter by name (replace `<process_name>` with the actual

name). The first column shows the PID. For Java processes, use `jps` or `jps -l` for a detailed listing.